

Q. What is DATA ?

Data is a raw-fact which describes the attributes of an Entity.

Q. What is DATABASE :

Database is a place or a medium in which we store the data in a Systematic and organized manner.

The basic operations that can be performed on a database are

- CREATE / INSERT
- READ / RETRIEVE
- UPDATE / MODIFY
- DELETE / DROP

Q. What is DBMS?

It is a software which is used to maintain and manage The database.

Security and authorization are the two important features that DBMS provides.

- We use query language to communicate or interact with DBMS
- DBMS stores the data in the form of files .

Q.What is RELATIONAL DATABASE MANAGEMENT SYSTEM(RDBMS)?

It is a type of DBMS software in which we store the data In the form of Tables (rows & columns) .

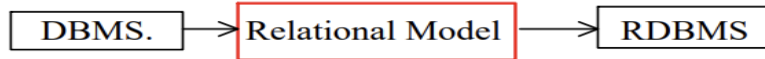
- We use SQL to communicate or interact with RDBMS
- RDBMS stores the data in the form of Tables.

RELATIONAL MODEL :

Relational Model was designed by **E.F CODD** .

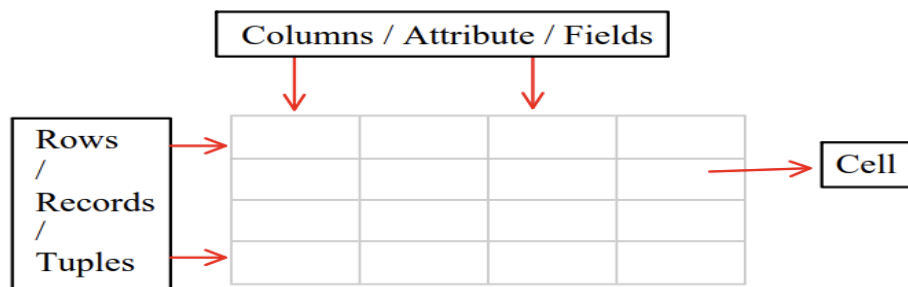
In Relational Model we can store the data in the form of *tables* .

Any DBMS which follows Relational Model becomes RDBMS .



Any DBMS which follows rules of EF CODD becomes RDBMS .

TABLE : "It is a logical organization of data which consists of Columns & Rows .



Q. What is DATATYPES ?

It is used to specify or determine the type of data that will be stored in a particular memory location.

Datatypes in SQL :

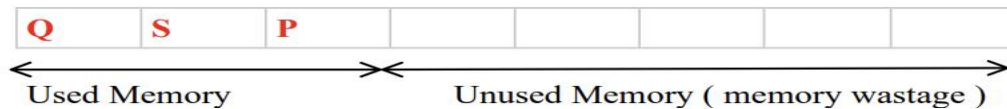
1. CHAR
2. VARCHAR / VARCHAR2
3. DATE
4. NUMBER LARGE OBJECTS
 - i. Character Large Object .
 - ii. Binary Large Object

1. **CHAR** : In character datatype we can store 'A-Z' , 'a-z' , '0-9' And Special Characters(\$, & , @ , ! ...) .

- Characters must always be enclosed within single quotes ''.
- Whenever we use char datatype we must mention size
- **Size** : it is used to specify number of characters it can store .
 - The maximum number of characters it can store is **2000ch.**
- Char follows fixed length memory allocation .

Syntax: CHAR (SIZE)

Example : CHAR (8)

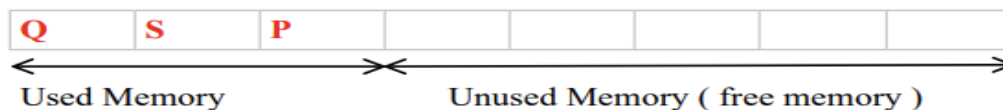


2. **VARCHAR** : In varchar datatype we can store 'A-Z' , 'a-z' , '0-9' And Special Characters(\$, & , @ , ! ...) .

- Characters must always be enclosed within single quotes ''.
- Whenever we use char datatype we must mention size
- **Size** : it is used to specify number of characters it can store .
 - The maximum number of characters it can store is **2000ch.**
- VarChar follows variable length memory allocation .

Syntax: VARCHAR (SIZE)

Example : VARCHAR (8)



NOTE : **VARCHAR2** : it is an updated version of varchar where in We can store up to **4000Ch.**

Syntax: VARCHAR2(SIZE)

3. **NUMBER** : It is used to store numeric values .

SYNTAX: NUMBER (Precision , [Scale])

[] - Not Mandatory .

Precision : it is used to determine the number of digits used To store integer value .

Scale : it is used to determine the number of digits used to store Decimal (floating) value within the precision .

- Scale is not mandatory , and the default value of scale Is zero (0) .

5. LARGE OBJECTS

1. Character large object (CLOB) :

It is used to store characters up to 4 GB of size .

2. Binary large object (BLOB) :

It is used to store binary values of images , mp3 , mp4 Documents etc Up to 4GB of size .

Q.What is CONSTRAINTS :-

It is a rule given to a column for validation .

Types of Constraints :

1. UNIQUE
2. NOT NULL
3. CHECK
4. PRIMARY KEY
5. FOREIGN KEY .

1. **UNIQUE** : *"It is used to avoid duplicate values into the column "*.
2. **NOT NULL** : *"It is used to avoid Null "*.
3. **CHECK** : *"It is an extra validation with a condition
If the condition is satisfied then the value is accepted else
Rejected "*.
4. **PRIMARY KEY** : *"It is a constraint which is used to identify a record
Uniquely from the table " .*

Characteristics of Primary key :

- We can have only 1 PK in a table
- PK cannot accept duplicate / repeated values .
- PK cannot accept Null
- PK is always a combination of Unique and Not Null Constraint.

5. **FOREIGN KEY** : *"It is used to establish a connection between the
The tables "*

Characteristics of Foreign key :

- We can have only Multiple FK in a table
- FK can accept duplicate / repeated values .
- FK can accept Null
- FK is not a combination of Unique and Not Null Constraint.
- For an Attribute (column) to become a FK ,it is mandatory
That it must be a PK in its own table .

<u>PRIMARY KEY</u>	<u>FOREIGN KEY</u>
It is used to identify a records Uniquely from the table.	It is used to establish a connection Between the tables
It cannot accept Null	It can accept Null
It cannot accept duplicate values	It can accept duplicate values
It is always a combination of Not Null and Unique constraint	It is not a combination of Not Null and Unique constraint
We can have only 1 PK in a table	We can have Multiple FK in a table

OVERVIEW OF SQL STATEMENTS :

1. DATA DEFINITION LANGUAGE (DDL)
2. DATA MANIPULATION LANGUAGE (DML)
3. TRANSCATION CONTROL LANGUAGE (TCL)
4. DATA CONTROL LANGUAGE (DCL)
5. DATA QUERY LANGUAGE (DQL)

DATA QUERY LANGUAGE (DQL) :

" DQL is used to retrieve the data from the database " .

It had 4 statements :

1. SELECT
2. PROJECTION
3. SELECTION
4. JOIN

1. **SELECT** : "It is used to retrieve the *data* from the table and display it.
2. **PROJECTION** : "It is a process of retrieving the data by *selecting only the columns* is known as Projection " .
 - In projection all the records / values present in a particular column are by default selected .
3. **SELECTION** : "It is a process of retrieving the data by *selecting both the columns and rows* is known as Selection " .
4. **JOIN** : "It is a process of retrieving the data from *Multiple tables* simultaneously is known as Join " .

Q. What is DISTINCT ?

It is used to remove the duplicate or repeated values from the Result table.

➔ Distinct clause has to be used As the first argument to select clause .

➤ We can use multiple columns As an argument to distinct clause, it will remove the combination of columns in which the records are duplicated .

Q.What is EXPRESSION ?

A statement which gives result is known as Expression.

Expression is a combination Operand and Operator .

ALIAS

"It is an alternate name given to a Column or an Expression In the result table " .

- We can assign alias name with or without using 'As' keyword .
- Alias names have to be a single string which is separated by An underscore or enclosed within double quotes .

Example :	ANNUAL_SALARY
	"ANNUAL SALARY"

OPERATORS IN SQL

1. ARITHMETIC OPERATORS :- (+ , - , * , /)
2. CONCATENATION OPERATOR :- (||)
3. COMPARISON OPERATORS :- (= , != or <>)
4. RELATIONAL OPERATOR :- (> , < , >= , <=)
5. LOGICAL OP : (**AND** , **OR** , **NOT**)
6. SPECIAL OPERATOR :-

1. **IN**
2. **NOT IN**
3. **BETWEEN**
4. **NOT BETWEEN**
5. **IS**
6. **IS NOT**
7. **LIKE**
8. **NOT LIKE**

7. SUBQUERY OPERATORS:-

1. ALL
2. ANY
3. EXISTS
4. NOT EXISTS

CONCATENATION Operator :

" It is used to join the strings "

Symbol : ||

```
SELECT 'Hi ' || ename
FROM EMP
WHERE JOB ='MANAGER' ;
```

<u>Ename</u>
Hi ALLEN
Hi MARTIN
Hi SMITH

1. **IN** : *It is a multi-valued operator which can accept multiple values At the RHS .*

Syntax: Column_Name / Exp **IN** (v1 , v2 , . . Vn)

2. **NOT IN :** *It is a multi-valued operator which can accept multiple values At the RHS . It is similar to IN op instead of selecting it Rejects the values .*

Syntax: Column_Name / Exp **NOT IN** (v1 , v2 , . . vn)

3. **BETWEEN :** *"It is used whenever we have range of values "*
[Start value and Stop Value] .

Syntax:

Column_Name BETWEEN **Lower_Range** AND **Higher_Range** ;

- ***Between Op works including the range .***

Example :

- WAQTD name and salary of the employees if the emp is earning Salary in the range 1000 to 3000 .

```
SELECT ENAME , SAL
FROM EMP
WHERE SAL BETWEEN 1000 AND 3000 ;
```

4. **NOT BETWEEN :** It is Opposite of Between .

Syntax:

Column_Name NOT BETWEEN Lower_Range AND Higher_Range ;

Example :

- WAQTD name and salary of the employees if the emp is not earning Salary in the range 1000 to 3000 .

```
SELECT ENAME , SAL
FROM EMP
WHERE SAL NOT BETWEEN 1000 AND 3000 ;
```

5. **IS :** *"It is used to compare only NULL "*

Syntax: Column_Name **IS** NULL ;

6. **IS NOT** : *"It is used to compare the values with NOT NULL "*.

Syntax: Column_Name **IS NOT** NULL ;

7. **LIKE** : *"It is used for Pattern Matching "*.

To achieve pattern matching we use special characters .

- Percentile (%)
- Underscore (_)

Syntax: Column_Name **LIKE** 'pattern' ;

Example :

- WAQTD details of the employee who's name starts with 'S' .

```
SELECT *  
FROM EMP  
WHERE ENAME LIKE 'S%' ;
```

- WAQTD details of the employee who's name ends with 'S' .

```
SELECT *  
FROM EMP  
WHERE ENAME LIKE '%S' ;  
  
WHERE ENAME LIKE '%S%' ;
```

- WAQTD names that starts with 'J' and ends with 'S' .

```
SELECT ENAME  
FROM EMP  
WHERE ENAME LIKE 'J%S' ;
```

- WAQTD names of the employee if the emp has char 'A' as his second character .

```
SELECT ENAME  
FROM EMP  
WHERE ENAME LIKE '_A%' ;
```

8. **NOT LIKE** : Opposite of Like .

Syntax: Column_Name **NOT LIKE** 'pattern' ;

Types of Functions in SQL :

1. *SINGLE ROW FUNCTIONS*
2. *MUTLI ROW FUNCTIONS / AGGREGATE / GROUP FUNCTIONS.*

Multi Row Functions:

It takes all the inputs at one shot and then executes and provides
A single output .

- If we pass 'n' number of inputs to a MRF() it returns '1' Output .

List of MRF ()

1. MAX() : it is used to obtain the maximum value present in the column
2. MIN() : it is used to obtain the minimum value present in the

- column
3. SUM() : it is used to obtain the summation of values present in the column
4. AVG() : it is used to obtain the average of values present in the column
5. COUNT() : it is used to obtain the number of values present in the column

NOTE :

- Multi row functions can accept only one argument , i.e a Column_Name or an Expression

MRF (Column_Name / Exp)

- Along with a MRF() we are not supposed to use any other Column_Name in the select clause .
- MRF() ignore the Null .
- We cannot use a MRF() in where clause .
- COUNT() is the only MRF which can accept * as an Argument .

10. WAQTD maximum salary given to a manager working in dept 20 and also his comm must be greater than his salary .

```
SELECT MAX( SAL )  
FROM EMP  
WHERE JOB ='MANAGER' AND DEPTNO = 20 AND COMM >  
SAL ;
```

GROUP & FILTERING

GROUPING : GROUP BY Clause

Group by clause is used to *group the records* .

SYNTAX:

```
SELECT group_by_expression / group_function  
FROM table_name  
[WHERE <filter_condition>]  
GROUP BY column_name/expression ;
```

ORDER OF EXECUTION:

```
1-FROM  
2-WHERE(if used) [ROW-BY-ROW]  
3-GROUP BY [ROW-BY-ROW]  
4-SELECT [GROUP-BY-GROUP]
```

NOTE :

- Group By clause is used to group the records .
- Group By clause executes row by row .
- After the execution of Group By clause we get Groups .
- Therefore any clause that executes after group by must execute Group By Group .
- The Column_Name or expression used for grouping can be used In select clause .
- Group By clause can be used without using Where clause .

2. WAQTD maximum salary given to each job .

```
SELECT JOB , MAX( SAL )  
FROM EMP  
GROUP BY JOB ;
```

3. WAQTD number of employees working in each job if the employees Have character 'A' in their names .

```
SELECT JOB , COUNT(*)  
FROM EMP  
WHERE ENAME LIKE '%A%'  
GROUP BY JOB ;
```

FILTERING : HAVING Clause

" Having Clause is used to Filter the Group "

SYNTAX:

```
SELECT group_by_expression / group_function  
FROM table_name  
[WHERE <filter_condition>]  
GROUP BY column_name/expression  
HAVING <group_filter_condition>
```

ORDER OF EXECUTION:

```
1-FROM  
2-WHERE(if used)      [ROW-BY-ROW]  
3-GROUP BY(if used)   [ROW-BY-ROW]  
4-HAVING (if used )   [GROUP-BY-GROUP]  
5-SELECT               [GROUP-BY-GROUP]
```

4. WAQTD the salary that is repeated .

```
SELECT SAL, COUNT(*)  
FROM EMP  
GROUP BY SAL  
HAVING COUNT(*) > 1 ;
```

5. WAQTD number of employees working in each dept having
At least 2 emp's Character 'A' or 'S' in their names .

```
SELECT DEPTNO , COUNT(*)  
FROM EMP  
WHERE ENAME LIKE '%A%' OR ENAME LIKE '%S%'  
GROUP BY DEPTNO  
HAVING COUNT(*)>=2 ;
```

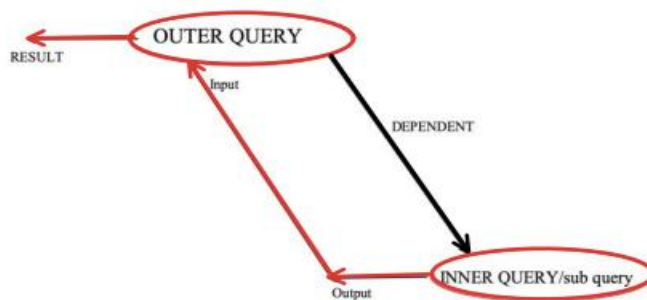
Differentiate between Where and Having .

<u>WHERE</u>	<u>HAVING</u>
➤ Where clause is used to Filter the records	➤ Having clause is used to Filter the groups .
➤ Where clause executes row By row .	➤ Having clause executes Group by group
➤ In Where Clause we cannot Use MRF()	➤ Can use MRF().
➤ Where clause executes before Group by clause .	➤ Having clause executes After group by clause .

SUB-QUERY

" A QUERY WRITTEN INSIDE ANOTHER QUERY IS KNOWN AS SUB QUERY "

Working Principle :



Let us consider two queries Outer Query and Inner Query .

- Inner Query executes first and produces an Output .
- The Output of Inner Query is given / fed as an Input to Outer Query .
- The Outer Query generates the Result.
- Therefore we can state that 'the Outer Query is dependent on Inner Query' and this is the Execution Principle of Sub Query .

4. WAQTD name and hiredate of the employees if the employee Was hired after JONES .

```

SELECT ENAME , HIREDATE
FROM EMP
WHERE HIREDATE > ( SELECT HIREDATE
                   FROM EMP
                   WHERE ENAME ='JONES' ) ;

```

5. WAQTD all the details of the employee working in the same Designation as KING .

```

SELECT *
FROM EMP
WHERE JOB = ( SELECT JOB
             FROM EMP
             WHERE ENAME ='KING' );

```

CASE-2 : Whenever the data to be selected and the condition to be executed are present in different tables we use Sub Query .

Example :

Emp

EID	ENAME	SAL	DEPTNO
1	ALLEN	1000	20
2	BLAKE	2000	10
3	CLARK	3000	30
4	MILLER	1500	10
5	ADAMS	2500	20

DEPT

DEPTNO	DNAME	LOC
10	D1	L1
20	D2	L2
30	D3	L3

1. WAQTD deptno of the employee whose name is Miller .

```
SELECT DEPTNO
FROM EMP
WHERE ENAME ='MILLER' ;
```

2. WAQTD **dname** of the employee whose name is **Miller** .

```
SELECT DNAME
FROM DEPT
WHERE DEPTNO = ( SELECT DEPTNO
                  FROM EMP
                  WHERE ENAME ='MILLER' ) ;
```

3. WAQTD Location of ADAMS

```
SELECT LOC
FROM DEPT
WHERE DEPTNO = ( SELECT DEPTNO
                  FROM EMP
                  WHERE ENAME ='ADAMS' ) ;
```

7. WAQTD all the details of the employee working as a Manager
In the dept Accounting .

```
SELECT *
FROM EMP
WHERE JOB ='MANAGER' AND
DEPTNO = ( SELECT DEPTNO
           FROM DEPT
           WHERE DNAME ='ACCOUNTING' ) ;
```


38. WAQTD DNAME AND LOC OF THE EMPLOYEES WHOS SALARY IS RUPEES 800 .

```
SELECT DNAME , LOC  
FROM DEPT  
WHERE DEPTNO = ( SELECT DEPTNO  
FROM EMP  
WHERE SAL = 800) ;
```

39. WAQTD DNAME OF THE EMPLOYEES WHO EARN COMMISSION

```
SELECT DNAME  
FROM DEPT  
WHERE DEPTNO = ( SELECT DEPTNO  
FROM EMP  
WHERE COMM IS NOT NULL );
```

TYPES OF SUB - QUERY :

1. SINGLE ROW SUB QUERY
2. MULTI ROW SUB QUERY

1. SINGLE ROW SUB QUERY :

- If the sub query returns exactly 1 record / value we call it as Single Row Sub Query .
- If it returns only 1 value then we can use the normal operators Or the Special Operators to compare the values .

2. MULTI ROW SUB QUERY :

- If the sub query returns more than 1 record / value we call it as Multi Row Sub Query .
- If it returns more than 1 value then we **cannot use the normal operators** We have to use only Special Operators to compare the values .

Note : It is difficult to identify whether a query Belongs Single or Multi row So , it is always recommended to use Special Operators to Compare The values .

2. WAQTD dnames of allen and smith .

```
SELECT DNAME
FROM DEPT
WHERE DEPTNO
```

DEPTNO
20
10
30
10
10

20
10

```
= ( SELECT DEPTNO
    FROM EMP
    WHERE ENAME IN
    ( 'ALLEN' , 'SMITH' ) );
```

Here , since the sub query returns 2 records we cannot use '=' Op . We've to use IN Op .

1. WAQTD ename and salary of the employees earning *more than* Employees of dept 10 .

EID	ENAME	SAL	DEPTNO
1	ALLEN	1000	20
2	BLAKE	2000	10
3	CLARK	3000	30
4	MILLER	1500	10
5	SMITH	2500	10

```
SELECT ENAME , SAL
FROM EMP
WHERE SAL
```

2000
1500
2500

```
> ( SELECT SAL
    FROM EMP
    WHERE DEPTNO = 10 );
```

Here we cannot use > symbol to compare Multiple values .

We cant use IN or. NOT IN as well because It is used for = and != symbols .

Therefore we have to use **Sub Query Operators** For Comparing Relational Operators such as (> , < , >= , <=) .

Sub Query Operators :

1. ALL :

"It is special Op used along with a relational Op ($>$, $<$, $>=$, $<=$) to compare the values present at the RHS ".

- *ALL Op returns true if all the values at the RHS have satisfied the condition .*

2. ANY :

"It is special Op used along with a relational Op ($>$, $<$, $>=$, $<=$) to compare the values present at the RHS ".

- *ANY Op returns true if one of the values at the RHS have satisfied the condition .*

2. WAQTD name of the employee if the employee earns less than At least a salesman .

```
SELECT ENAME
FROM EMP
WHERE SAL < ANY ( SELECT SAL
FROM EMP
WHERE JOB ='SALESMAN' ) ;
```

3. WAQTD names of the employees earning more than ADAMS .

```
SELECT ENAME
FROM EMP
WHERE SAL > ALL ( SELECT SAL
FROM EMP
WHERE ENAME ='ADAMS' ) ;
```

JOINS

"The process of retrieval of data from multiple tables simultaneously is known as JOINS "

Types of JOINS .

We have 5 types of joins

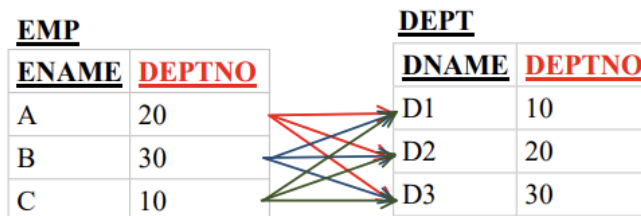
1. CARTESIAN JOIN / CROSS JOIN
2. INNER JOIN / EQUI JOIN
3. OUTER JOIN
 - i. LEFT OUTER JOIN
 - ii. RIGHT OUTER JOIN
 - iii. FULL OUTER JOIN
4. SELF JOIN
5. NATURAL JOIN .

1. CARTESIAN JOIN / CROSS JOIN :

In Cartesian Join a record from table 1 will be merged with All the records of table 2 .

2. INNER JOIN :

"It is used to Obtain only Matching Records "
Or " A records which has a Pair " .



JOIN Condition : It is a condition on which the two tables
Are merged .

Syntax: Table _Name1.Col _Name = Table _Name2.Col _Name

OUTER JOIN

"It is used to Obtain Un-Matched Records "

1. Left Outer Join :

"It is used to obtain Un-Matched Records of Left Table Along with Matching Records " .

2. **Right Outer Join :**

"It is used to obtain Un-Matched Records of Right Table Along with Matching Records "

SELF JOIN :

"Joining a table by itself is known as Self Join "

Why ? / When ?

"Whenever the data to select is in the same table but present In different records we use self-join "

5. **NATURAL JOIN :**

"It behaves as INNER JOIN if there is a relation between the given two tables , else it behaves as CROSS JOIN" .

Syntax:

ANSI :

```
SELECT Col_Name  
FROM Table_Name1 NATURAL JOIN Table_Name2;
```

To Find MAX & MIN salary :

To find MAXIMUM salary :

```
SELECT SAL
FROM EMP E1
WHERE ( SELECT COUNT( DISTINCT SAL )
        FROM EMP E2
        WHERE E1.SAL < E2.SAL ) = N-1 ;
```



To find MINIMUM salary :

```
SELECT SAL
FROM EMP E1
WHERE ( SELECT COUNT( DISTINCT SAL )
        FROM EMP E2
        WHERE E1.SAL > E2.SAL ) = N-1 ;
```

SINGLE ROW FUNCTIONS

1. LENGTH()
2. CONCAT()
3. UPPER()
4. LOWER()
5. INITCAP()
6. REVERSE()
7. SUBSTR()
8. INSTR()
9. REPLACE()
10. MOD()
11. TRUNC()
12. ROUND()
13. MONTHS_BETWEEN()
14. LAST_DAY()
15. TO_CHAR()
16. NVL()

SYNTAX : CONCAT ('string1' , 'String2')

Example :

Input : Smith

Output : Mr. Smith

```
SELECT CONCAT( 'Mr. ' , ENAME )
FROM EMP
WHERE ENAME = 'SMITH' ;
```

3. **UPPER()** : "It is used to convert a given string to upper case "

SYNTAX: UPPER ('string')

4. **LOWER()** : "It is used to convert a given string to lower case "

SYNTAX: LOWER('string')

5. **INITCAP()** : "It is used to convert a given string to initial capital letter case ".

SYNTAX: INITCAP('string')

6. **REVERSE()** : "It is used to reverse a given string ".

SYNTAX: REVERSE('string')

7. **SUBSTR** : "It is used to extract a part of string from the given Original string " .

SYNTAX: **SUBSTR** ('Original_String' , Position [, Length])

Example :

NOTE: Length is not mandatory ,
If length is not mentioned then

8. **REPLACE ()** : "It is used to replace a string with another string in The original string.

SYNTAX: **REPLACE** ('Original_String' , 'string' [, 'new_String'])

Null

9. **INSTR ()** : "it is used to obtain the position in which the string is present in the Original string ".
It is used to search for a string in the Original string if present it returns the POSITION
Else it returns 0 ".

Syntax: **INSTR**('Original_String' , 'String' , Position [, Occurrence])

Note : if occurrence is not Mentioned then , the default value of Occurrence is 1 .

B	A	N	A	N	A
1	2	3	4	5	6

Example : INSTR('BANANA' , 'A' , 1 , 1)	POS: 2
Example : INSTR('BANANA' , 'A' , 2 , 1)	POS: 2
Example : INSTR('BANANA' , 'A' , 1 , 2)	POS: 4
Example : INSTR('BANANA' , 'A' , 1 , 3)	POS: 6
Example : INSTR('BANANA' , 'A' , 1 , 4)	POS: 0
Example : INSTR('BANANA' , 'A' , 4 , 2)	POS: 6
Example : INSTR('BANANA' , 'A' , 2)	POS: 2
Example : INSTR('BANANA' , 'N' , 2 , 1)	POS: 3
Example : INSTR('BANANA' , 'O' , 1 , 1)	POS: 0
Example : INSTR('BANANA' , 'NA' , 2 , 2)	POS: 5
Example : INSTR('BANANA' , 'A' , 3 , 3)	POS: 0
Example : INSTR('BANANA' , 'ANA' , 1 , 2)	POS: 4

Q. What is the difference between DBMS and RDBMS?

Software systems used to store and manage data, but they differ in how they organize and handle that data.

Feature	DBMS	RDBMS
Definition	A system for storing and managing data.	A DBMS based on the relational model.
Data Storage	Data may be stored as files or hierarchical structures.	Data is stored in tables (rows and columns).
Relationships	Does not necessarily support relationships between data.	Supports relationships using primary keys and foreign keys.
Normalization	Usually does not support normalization.	Supports normalization to reduce data redundancy.
Data Redundancy	Higher redundancy.	Lower redundancy due to normalization.
Security	Limited security features.	Strong security and access control mechanisms.
Multi-user Access	Limited support.	Designed for multiple users simultaneously.
ACID Properties	May not fully support ACID transactions.	Supports ACID properties (Atomicity, Consistency, Isolation, Durability).
Examples	dBase, FoxPro, IMS	Oracle Database , MySQL , PostgreSQL , Microsoft SQL Server

Q. What is a unique key?

A **Unique Key** is a constraint in SQL that ensures all values in a column are unique. It does not allow duplicate values in that column. A table can have multiple unique keys.

Q: Difference between Primary Key and Unique Key?

Primary Key: Cannot contain NULL values, and there can be only one primary key in a table.

Unique Key: Prevents duplicate values and can allow NULL values (database-dependent). A table can have multiple unique keys.

Q. What is a composite key?

A **Composite Key** is a key that is created by combining **two or more columns** to uniquely identify each record in a table.

Example

`</> SQL`

```
CREATE TABLE Student_Course (  
  StudentID INT,  
  CourseID INT,  
  PRIMARY KEY (StudentID, CourseID)  
);
```

Here:

- `StudentID` alone is not unique.
- `CourseID` alone is not unique.
- But the combination of `StudentID` and `CourseID` is unique.

Q. What is normalization?

Normalization is the process of organizing data in a database to **reduce data redundancy (duplicate data)** and **improve data integrity**.

Why Normalization?

- Removes duplicate data.
- Saves storage space.
- Makes data maintenance easier.
- Improves data consistency.

Types of Normalization

- **1NF (First Normal Form)** – No repeating groups, atomic values.
- **2NF (Second Normal Form)** – Remove partial dependency.
- **3NF (Third Normal Form)** – Remove transitive dependency.
- **BCNF** – Advanced form of 3NF.

Q: Difference between INNER JOIN and OUTER JOIN?

INNER JOIN

Returns only matching records from both tables.

Excludes unmatched rows.

Most commonly used join.

OUTER JOIN

Returns matching records and also non-matching records.

Includes unmatched rows with NULL values.

Includes LEFT JOIN, RIGHT JOIN, and FULL JOIN.

INNER JOIN

 SQL

```
SELECT e.Name, d.Department
FROM Employee e
INNER JOIN Department d
ON e.EmpID = d.EmpID;
```

LEFT OUTER JOIN

 SQL

```
SELECT e.Name, d.Department
FROM Employee e
LEFT JOIN Department d
ON e.EmpID = d.EmpID;
```

Q. What are LEFT JOIN and RIGHT JOIN?

LEFT JOIN

Definition:

A **LEFT JOIN** returns **all records from the left table** and the matching records from the right table. If there is no match, NULL values are returned for the right table columns.

 SQL

```
SELECT *
FROM Employee e
LEFT JOIN Department d
ON e.EmpID = d.EmpID;
```

RIGHT JOIN

Definition:

A **RIGHT JOIN** returns **all records from the right table** and the matching records from the left table. If there is no match, NULL values are returned for the left table columns.

 SQL

```
SELECT *
FROM Employee e
RIGHT JOIN Department d
ON e.EmpID = d.EmpID;
```

Q.What is SELF JOIN?

A **SELF JOIN** is a join in which a table is joined with **itself**. It is used when we want to compare rows within the same table.

Why Use SELF JOIN?

- Employee-Manager relationships
- Finding duplicate records
- Comparing rows within the same table

<code></> SQL</code>	
<pre>SELECT e.Name AS Employee, m.Name AS Manager FROM Employee e LEFT JOIN Employee m ON e.ManagerID = m.EmpID;</pre>	
Output	
Employee	Manager
Rahul	NULL
Amit	Rahul
Neha	Rahul

Q. Difference between JOIN and UNION?

Q: Difference Between JOIN and UNION?	
JOIN	UNION
Combines columns from multiple tables.	Combines rows from multiple queries.
Uses a common column to relate tables.	Does not require a common column.
Result has more columns.	Result has more rows.
Types: INNER, LEFT, RIGHT, FULL JOIN.	Types: UNION, UNION ALL.

<code></> SQL</code>
<pre>SELECT e.Name, d.Department FROM Employee e INNER JOIN Department d ON e.EmpID = d.EmpID;</pre>

<code></> SQL</code>
<pre>SELECT Name FROM Table1 UNION SELECT Name FROM Table2;</pre>

Q. Difference between DELETE, DROP, and TRUNCATE?

Q: Difference between DELETE, DROP, and TRUNCATE?			
DELETE	TRUNCATE	DROP	
Removes selected rows from a table.	Removes all rows from a table.	Removes the entire table structure and data.	
Uses <code>WHERE</code> clause.	Cannot use <code>WHERE</code> clause.	No <code>WHERE</code> clause.	
Table structure remains.	Table structure remains.	Table structure is deleted.	
Can be rolled back (in transactions).	Usually cannot be rolled back.	Usually cannot be rolled back.	
Slower.	Faster than DELETE.	Fastest.	

Q: Difference Between UNION and UNION ALL?

UNION	UNION ALL
Removes duplicate rows.	Keeps duplicate rows.
Slower because it checks for duplicates.	Faster because it does not check duplicates.
Returns only unique records.	Returns all records, including duplicates.

Q. Difference between GROUP BY and ORDER BY?

GROUP BY is used to group rows having the same values, while **ORDER BY** is used to sort the result set in ascending or descending order.

GROUP BY Example

Find the number of employees in each department:

 SQL

```
SELECT Department, COUNT(*)  
FROM Employee  
GROUP BY Department;
```

ORDER BY Example

Display employees sorted by salary:

 SQL

```
SELECT *  
FROM Employee  
ORDER BY Salary DESC;
```

 Sorts records from highest to lowest salary.

Easy Trick

- **GROUP BY** → **Group** similar records.
- **ORDER BY** → **Order** (sort) records.

Q: Difference Between ANY and ALL in SQL?

ANY

Condition is true if it matches **at least one** value in the subquery.

Acts like "at least one".

Less restrictive.

ALL



Condition is true only if it matches **all** values in the subquery.

Acts like "every value".

More restrictive.

Q. What is a transaction?

A **Transaction** is a sequence of one or more SQL operations that are executed as a **single unit of work**.

A transaction ensures that either:

- **All operations are completed successfully**, or
- **None of them are applied** (rollback).

Why Use Transactions?

- Maintains data consistency.
- Prevents partial updates.
- Ensures data integrity.

- Useful in banking, e-commerce, and payment systems.

Q. What are ACID properties?

ACID properties ensure that database transactions are processed **reliably and safely**.

ACID = Atomicity + Consistency + Isolation + Durability

1. Atomicity (All or Nothing)

A transaction is treated as a single unit.

- Either all operations succeed.
- Or all operations fail.

2. Consistency

- The database must remain in a valid state before and after the transaction.

3. Isolation

- Multiple transactions should not interfere with each other.

Two users updating the same account at the same time should not see incomplete changes from each other.

4. Durability

Once a transaction is committed, the changes are permanent.

Example:

After a successful money transfer and COMMIT, the data remains saved even if the system crashes.

Q. What is COMMIT?

COMMIT is a transaction control command used to **save all changes permanently** in the database.

Once a transaction is committed:

- Changes are permanently stored.
- They cannot be undone using ROLLBACK.

Why Use COMMIT?

- Saves transaction changes permanently.
- Ensures data is stored successfully.
- Marks the successful completion of a transaction.

Q. What is ROLLBACK?

ROLLBACK is a transaction control command used to **undo changes** made during a transaction and restore the database to its previous state.

Example

 SQL 

```
BEGIN TRANSACTION;  
  
UPDATE Employee  
SET Salary = Salary + 5000  
WHERE EmpID = 101;  
  
ROLLBACK;
```

✅ The salary update is canceled, and the database returns to its previous state.

Why Use ROLLBACK?

- Undo incorrect changes.
- Handle transaction failures.
- Maintain data consistency.

Easy Trick

- **COMMIT** → Save Changes ✅
- **ROLLBACK** → Undo Changes ❌

Q. What is SAVEPOINT?

A **SAVEPOINT** is a point inside a transaction that allows you to **rollback to a specific point** instead of rolling back the entire transaction.

What Happens?

- First update is executed.
- SAVEPOINT sp1 is created.
- Second update is executed.
- ROLLBACK TO sp1 undoes only the second update.
- First update remains.
- COMMIT saves the remaining changes.

Why Use SAVEPOINT?

- Partial rollback of a transaction.
- Better control over transactions.

- Useful in complex transactions with multiple operations.